

Relational Learning with Covariance Information

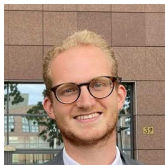
Elvin Isufi

`e.isufi-1@tudelft.nl`

November 28, 2025



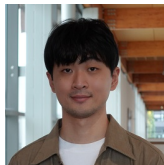
Acknowledgements



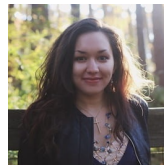
Andrea Cavallo



Mohammad Sabbaghi



Zhan Gao



Madeline Navarro



Santiago Segarra



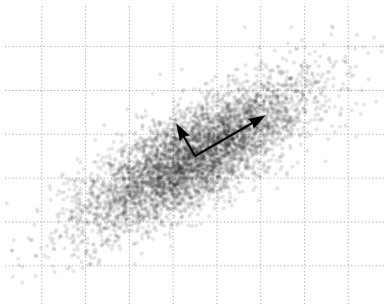
AI Initiative

- One way to capture these relations is through the **covariance matrix**
 $\Rightarrow \mathbf{C} = \mathbb{E}[(\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^\top]$



Principal Component Analysis (PCA)

- Project the data onto the **covariance eigenspace**
 - $\Rightarrow \mathbf{C} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^\top$
 - $\Rightarrow \tilde{\mathbf{x}} = \mathbf{V}^\top \mathbf{x}$
- Select (**filter**) the directions that **maximize the variance** of data points
 - $\Rightarrow$ Used for **dimensionality reduction** by selecting only a few eigenvectors

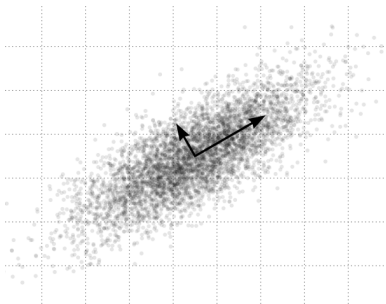


Principal directions

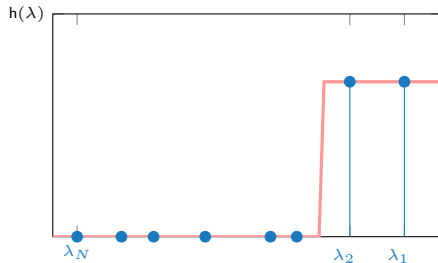


Principal Component Analysis (PCA)

- Project the data onto the **covariance eigenspace**
 - $\Rightarrow \mathbf{C} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^\top$
 - $\Rightarrow \tilde{\mathbf{x}} = \mathbf{V}^\top \mathbf{x}$
- Select (**filter**) the directions that **maximize the variance** of data points
 - $\Rightarrow$ Used for **dimensionality reduction** by selecting only a few eigenvectors



Principal directions

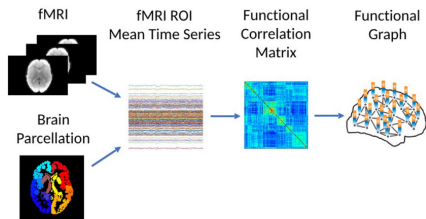


PCA filtering

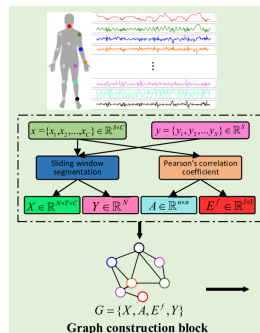


Topology Inference

- Graphical Lasso, Collaborative Filtering, Graph Stationarity



Brain fMRI data



Human motion sensor recordings

- Use this topology for downstream processing

Wang et al., MhaGNN, IEEE Transactions on Instrumentation and Measurement, 2023
Li et al., BrainGNN: Interpretable brain graph neural network for fMRI analysis. Medical Image Analysis, 2021

Learning with covariances

Finite data problems

- In practice, we work with estimates

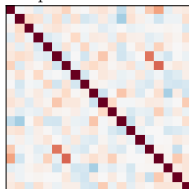
$$\Rightarrow \hat{\mathbf{C}} = \frac{1}{t} \sum_{t'=1}^t \mathbf{x}_{t'} \mathbf{x}_{t'}^{\top}$$

- May **not** reflect the **true** underlying structure

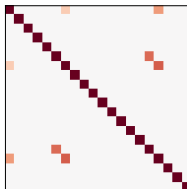
$\Rightarrow$ Sample estimator $\hat{\mathbf{C}}$ is noisy

$\Rightarrow$ When number of samples t is of same order of data dimension N

Empirical covariance

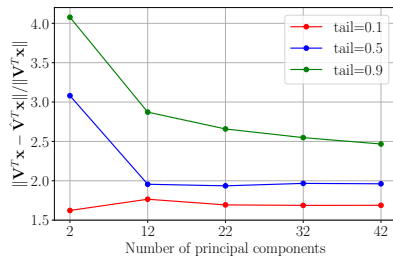


True covariance



Finite-data effect on PCA

- PCA is **unstable** to covariance estimation errors



Empirical stability

Johnstone & Lu. On consistency and sparsity for principal components analysis in high dimensions. JASA. 2009

Learning with covariances

- ▶ **Finite-data:** alterations in the principal directions
- ▶ **Sparse estimates:** alterations in the principal directions
- ▶ **Solution:** graph neural networks
 - ⇒ Covariance-based neural networks **improve stability**
 - ⇒ Can be made efficient through **sparsification**
 - ⇒ Work well in a **variety of settings**: static, temporal, and biased data



Outline

- ▶ Covariance Neural Networks
- ▶ Sparse VNNs
- ▶ Spatiotemporal VNNs
- ▶ Fair VNNs
- ▶ Conclusions



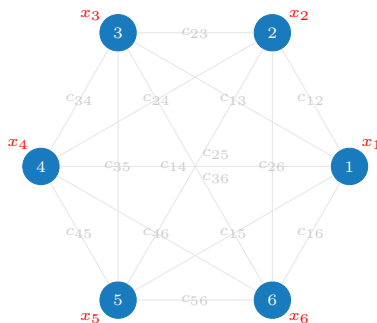
Outline

- ▶ **Covariance Neural Networks**
- ▶ Sparse VNNs
- ▶ Spatiotemporal VNNs
- ▶ Fair VNNs
- ▶ Conclusions



Covariance Data Relationships

- Data sample $\mathbf{x} = [x_1, \dots, x_N]^T$ with covariance $\mathbf{C}$
 - ⇒ Build a **graph** where:
 - ⇒ the features are the node signals x_i
 - ⇒ the edges are the covariance values c_{ij} → fully-connected graph



Covariance Filters

- Definition: Graph convolution covariance filters

$$\mathbf{z} = \mathbf{H}(\hat{\mathbf{C}})\mathbf{x} = \sum_{k=0}^K h_k \hat{\mathbf{C}}^k \mathbf{x}$$

⇒ learnable parameters: h_k

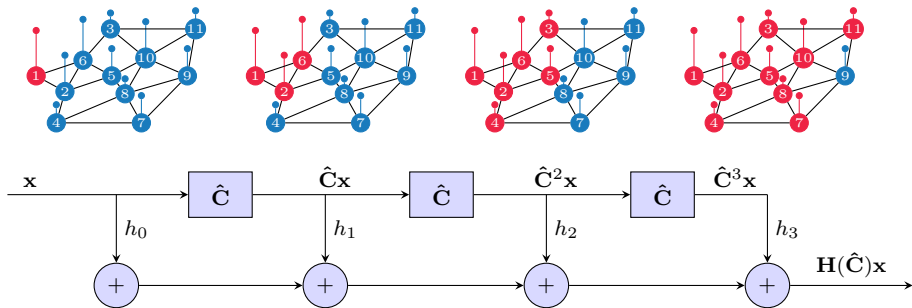


Covariance Filters

- **Definition:** Graph convolution covariance filters

$$\mathbf{z} = \mathbf{H}(\hat{\mathbf{C}})\mathbf{x} = \sum_{k=0}^K h_k \hat{\mathbf{C}}^k \mathbf{x}$$

⇒ learnable parameters: h_k



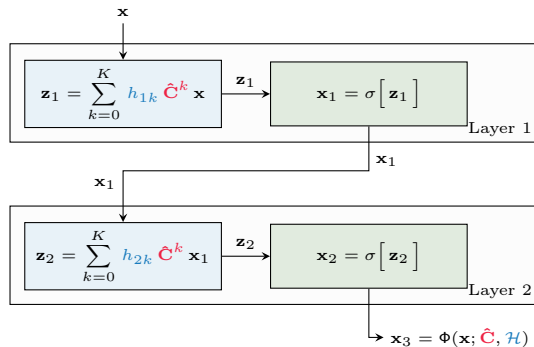
- $\hat{\mathbf{C}}^k \mathbf{x}$ shifts signal $\mathbf{x}$ k times over the covariance graph $\hat{\mathbf{C}}$



Covariance Neural Networks (VNNs)

- **Definition:** Covariance filters followed by pointwise nonlinearities σ

$$\mathbf{x}^l = \sigma \left(\mathbf{H}^l(\hat{\mathbf{C}}) \mathbf{x}^{l-1} \right) \quad l = 1, \dots, L.$$



Connections to PCA

- The covariance filter matrix has the form

$$\mathbf{H}(\hat{\mathbf{C}}) = \sum_{k=0}^K h_k \hat{\mathbf{C}}^k$$



Connections to PCA

- The covariance filter matrix has the form

$$\mathbf{H}(\hat{\mathbf{C}}) = \sum_{k=0}^K h_k \hat{\mathbf{C}}^k$$

- Taking the eigendecomposition $\hat{\mathbf{C}} = \hat{\mathbf{V}} \hat{\mathbf{\Lambda}} \hat{\mathbf{V}}^\top$ we get

$$\mathbf{H}(\hat{\mathbf{V}} \hat{\mathbf{\Lambda}} \hat{\mathbf{V}}^\top) = \sum_{k=0}^K h_k \hat{\mathbf{V}} \hat{\mathbf{\Lambda}}^k \hat{\mathbf{V}}^\top$$



Connections to PCA

- The covariance filter matrix has the form

$$\mathbf{H}(\hat{\mathbf{C}}) = \sum_{k=0}^K h_k \hat{\mathbf{C}}^k$$

- Taking the eigendecomposition $\hat{\mathbf{C}} = \hat{\mathbf{V}} \hat{\mathbf{\Lambda}} \hat{\mathbf{V}}^\top$ we get

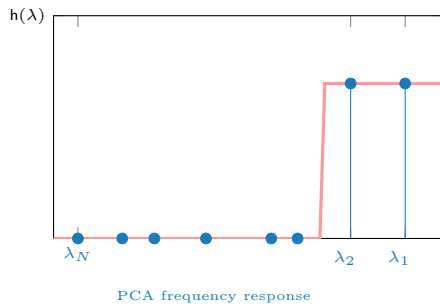
$$\mathbf{H}(\hat{\mathbf{V}} \hat{\mathbf{\Lambda}} \hat{\mathbf{V}}^\top) = \sum_{k=0}^K h_k \hat{\mathbf{V}} \hat{\mathbf{\Lambda}}^k \hat{\mathbf{V}}^\top$$

which means that in the spectrum it acts as a polynomial

$$h(\hat{\lambda}_i) = \sum_{k=0}^K h_k \hat{\lambda}_i^k$$

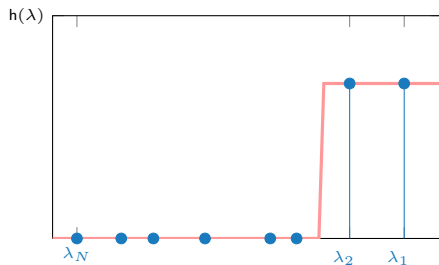
Connections to PCA

- PCA does a **hard** filtration by **selecting only** a few principal directions

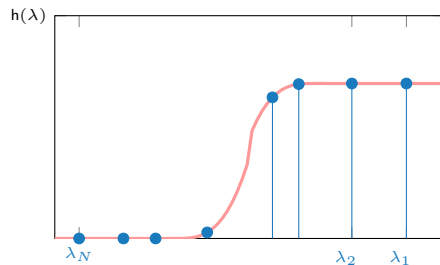


Connections to PCA

- PCA does a **hard** filtration by **selecting only** a few principal directions



PCA frequency response



VNN frequency response

- VNNs learn to **modulate** each principal direction to solve the task

Stability of VNNs

- ▶ We train the VNN on the noisy covariance matrix $\hat{\mathbf{C}}$
 $\Rightarrow$ How does this affect the performance?
- ▶ Stability for true $\mathbf{H}(\mathbf{C})$ vs. estimated $\mathbf{H}(\hat{\mathbf{C}})$ covariance filter:

$$\|\mathbf{H}(\hat{\mathbf{C}}) - \mathbf{H}(\mathbf{C})\| \leq \frac{P}{\sqrt{t}} \mathcal{O} \left(\sqrt{N} + \frac{\|\mathbf{C}\| \sqrt{\log(Nt)}}{\nu t} \right)$$

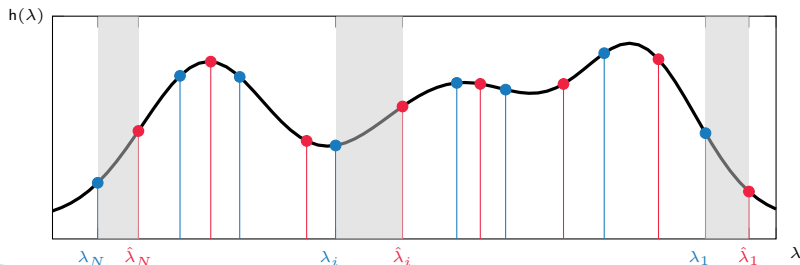


Stability of VNNs

- ▶ We train the VNN on the noisy covariance matrix $\hat{\mathbf{C}}$
 $\Rightarrow$ How does this affect the performance?
- ▶ Stability for true $\mathbf{H}(\mathbf{C})$ vs. estimated $\mathbf{H}(\hat{\mathbf{C}})$ covariance filter:

$$\|\mathbf{H}(\hat{\mathbf{C}}) - \mathbf{H}(\mathbf{C})\| \leq \frac{P}{\sqrt{t}} \mathcal{O} \left(\sqrt{N} + \frac{\|\mathbf{C}\| \sqrt{\log(Nt)}}{\nu t} \right)$$

- ▶ P : Lipschitz constant of the filter – variability of the frequency response



Stability of VNNs

- ▶ We train the VNN on the noisy covariance matrix $\hat{\mathbf{C}}$
 $\Rightarrow$ How does this affect the performance?
- ▶ Stability for true $\mathbf{H}(\mathbf{C})$ vs. estimated $\mathbf{H}(\hat{\mathbf{C}})$ covariance filter:

$$\|\mathbf{H}(\hat{\mathbf{C}}) - \mathbf{H}(\mathbf{C})\| \leq \frac{P}{\sqrt{t}} \mathcal{O} \left(\sqrt{N} + \frac{\|\mathbf{C}\| \sqrt{\log(Nt)}}{\nu t} \right)$$

- ▶ P : Lipschitz constant of the filter – variability of the frequency response
- ▶ t and N : number of samples and data dimension



Stability of VNNs

- ▶ We train the VNN on the noisy covariance matrix $\hat{\mathbf{C}}$
 $\Rightarrow$ How does this affect the performance?
- ▶ Stability for true $\mathbf{H}(\mathbf{C})$ vs. estimated $\mathbf{H}(\hat{\mathbf{C}})$ covariance filter:

$$\|\mathbf{H}(\hat{\mathbf{C}}) - \mathbf{H}(\mathbf{C})\| \leq \frac{P}{\sqrt{t}} \mathcal{O} \left(\sqrt{N} + \frac{\|\mathbf{C}\| \sqrt{\log(Nt)}}{\nu t} \right)$$

- ▶ P : Lipschitz constant of the filter – variability of the frequency response
- ▶ t and N : number of samples and data dimension
- ▶ Data distribution



Stability of VNNs

- Stability bound for covariance neural network:

$$\|\mathbf{H}(\hat{\mathbf{C}}) - \mathbf{H}(\mathbf{C})\| \leq \frac{P}{\sqrt{t}} \mathcal{O} \left(\sqrt{N} + \frac{\|\mathbf{C}\| \sqrt{\log(Nt)}}{\nu t} \right) = \beta$$

$$\|\Phi(\mathbf{x}, \hat{\mathbf{C}}, \mathcal{H}) - \Phi(\mathbf{x}, \mathbf{C}, \mathcal{H})\| \leq L F^{L-1} \beta$$

- ⇒ Bound for the **neural network** depends on the **filter bound**
- ⇒ Increases with the **number of layers** L and **filter bank size** F
- ⇒ The non-linearity spreads the energy across the spectrum

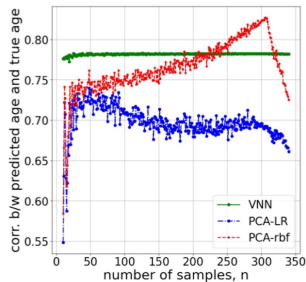
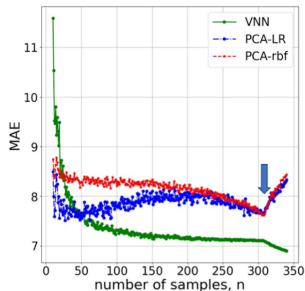


Covariance Neural Networks (VNNs)

Numerical results

► Setup:

- ⇒ Train with one covariance, test with another with different number of samples
- ⇒ **Brain dataset**: predict brain age
- ⇒ VNN vs PCA+SVM



- VNN performance is **more consistent**



Limitations of VNNs

- ▶ So are VNNs enough?

⇒ No

- ▶ Limitations

⇒ Low-data high-dimensional settings → sample covariance estimator is really bad!

⇒ Computationally inefficient → complexity $\mathcal{O}(N^2)$



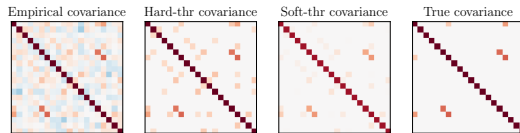
Outline

- ▶ Covariance Neural Networks
- ▶ **Sparse VNNs**
- ▶ Spatiotemporal VNNs
- ▶ Fair VNNs
- ▶ Conclusions



Sparse Covariance Matrices

- In **practice**, the **true** covariance matrix **C** is **sparse**



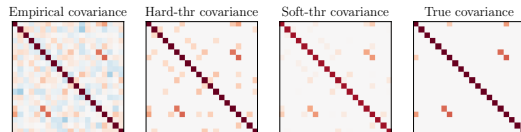
- Hard thresholding:

$$[\bar{\mathbf{C}}]_{ij} = \begin{cases} \hat{c}_{ij} & \text{if } |\hat{c}_{ij}| \geq \tau/\sqrt{t} \\ 0 & \text{otherwise} \end{cases}$$



Sparse Covariance Matrices

- In **practice**, the **true** covariance matrix **C** is **sparse**



- Hard thresholding:

$$[\bar{\mathbf{C}}]_{ij} = \begin{cases} \hat{c}_{ij} & \text{if } |\hat{c}_{ij}| \geq \tau/\sqrt{t} \\ 0 & \text{otherwise} \end{cases}$$

- Soft thresholding:

$$[\bar{\mathbf{C}}]_{ij} = \begin{cases} \hat{c}_{ij} - \text{sign}(\hat{c}_{ij})\tau/\sqrt{t} & \text{if } |\hat{c}_{ij}| \geq \tau/\sqrt{t} \\ 0 & \text{otherwise} \end{cases}$$



Sparse Covariance Neural Networks (S-VNNs)

- Stability for **true** $\mathbf{H}(\mathbf{C})$ vs. **hard-thresholded** $\mathbf{H}(\bar{\mathbf{C}})$ covariance filter:

$$\|\mathbf{H}(\bar{\mathbf{C}}) - \mathbf{H}(\mathbf{C})\| \leq \frac{1}{\sqrt{t}} P_{c_0} \sqrt{N \log N} (1 + \sqrt{2N}) + \mathcal{O}\left(\frac{1}{t}\right).$$

- c_0 : number of non-zero elements in $\bar{\mathbf{C}}$
 - $\Rightarrow c_0 \ll \|\mathbf{C}\| \rightarrow$ improved stability!
 - $\Rightarrow$ We need fewer data



Sparse Covariance Neural Networks (S-VNNs)

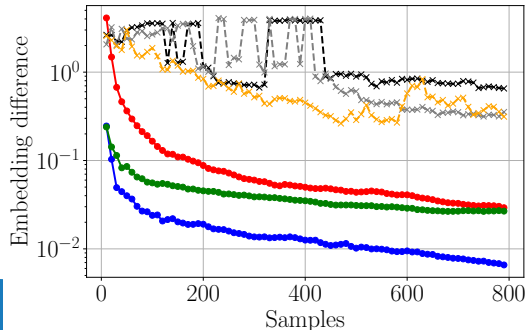
- Stability for **true $\mathbf{H}(\mathbf{C})$** vs. **hard-thresholded $\mathbf{H}(\bar{\mathbf{C}})$** covariance filter:

$$\|\mathbf{H}(\bar{\mathbf{C}}) - \mathbf{H}(\mathbf{C})\| \leq \frac{1}{\sqrt{t}} P c_0 \sqrt{N \log N} (1 + \sqrt{2N}) + \mathcal{O}\left(\frac{1}{t}\right).$$

- c_0 : number of non-zero elements in $\bar{\mathbf{C}}$

$\Rightarrow c_0 \ll \|\mathbf{C}\| \rightarrow$ improved stability!

$\Rightarrow$ We need fewer data



Sparse Covariance Neural Networks (S-VNNs)

- Stability for **true** $\mathbf{H}(\mathbf{C})$ vs. **soft-thresholded** $\mathbf{H}(\bar{\mathbf{C}})$ covariance filter:

$$\|\mathbf{H}(\bar{\mathbf{C}}) - \mathbf{H}(\mathbf{C})\| \leq \frac{1}{\sqrt{t}} P \sqrt{N} C c_0 \max(1, \lambda_{max}) \sqrt{\max(\log(N/c_0^2), 1)} (1 + \sqrt{2N}) + \mathcal{O}\left(\frac{1}{t}\right).$$

- $\sqrt{\max(\log(N/c_0^2), 1)} \leq \sqrt{\log N} \rightarrow$ further improved stability!

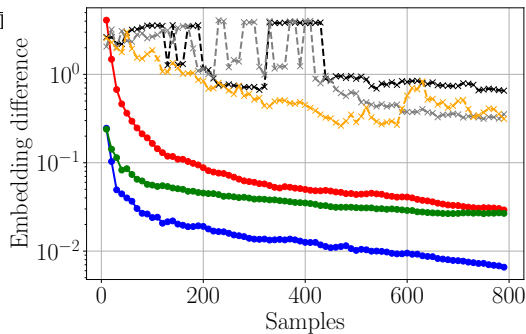


Sparse Covariance Neural Networks (S-VNNs)

- Stability for **true** $\mathbf{H}(\mathbf{C})$ vs. **soft-thresholded** $\mathbf{H}(\bar{\mathbf{C}})$ covariance filter:

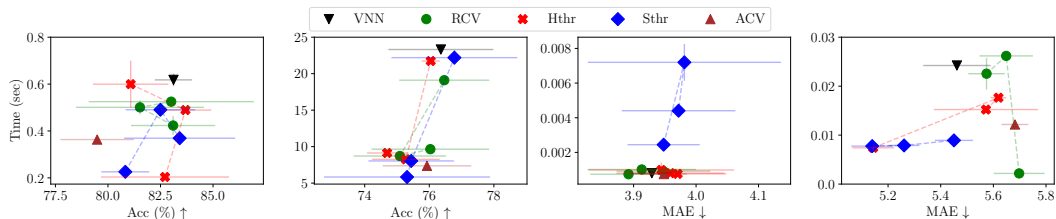
$$\|\mathbf{H}(\bar{\mathbf{C}}) - \mathbf{H}(\mathbf{C})\| \leq \frac{1}{\sqrt{t}} P \sqrt{N} C c_0 \max(1, \lambda_{\max}) \sqrt{\max(\log(N/c_0^2), 1)} (1 + \sqrt{2N}) + \mathcal{O}\left(\frac{1}{t}\right).$$

- $\sqrt{\max(\log(N/c_0^2), 1)} \leq \sqrt{1}$



Numerical Results

- Brain recordings: classify patient condition
- Human Action Recognition: classify action performed



Datasets: MHEALTH, Realdisp, ADNI1 and ADNI2

- S-VNNs are always faster and more accurate due to spurious correlation removal



Outline

- ▶ Covariance Neural Networks
- ▶ Sparse VNNs
- ▶ **Spatiotemporal VNNs**
- ▶ Fair VNNs
- ▶ Conclusions

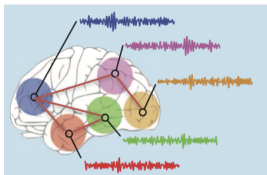


VNNs and Temporal Data

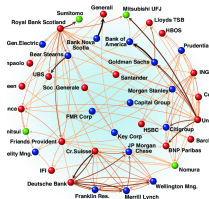
► So far:

⇒ VNNs and variants work well on static and batch data

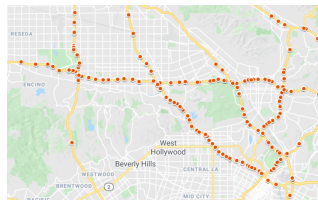
⇒ Yet, data are often time-varying and non-stationary



Brain



Financial



Traffic

► Goal: Incorporate spatiotemporal interdependencies

SpatioTemporal Covariance Neural Networks (STVNNs)

- Online covariance estimate

$$\hat{\mathbf{C}}_{t+1} = \xi_t \hat{\mathbf{C}}_t + \zeta_t \mathbf{x}_{t+1} \mathbf{x}_{t+1}^T$$



SpatioTemporal Covariance Neural Networks (STVNNs)

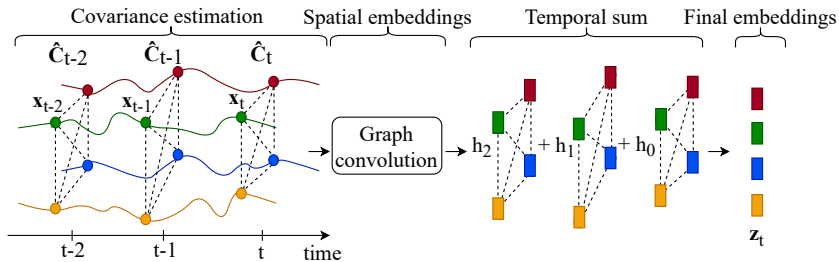
- Online covariance estimate

$$\hat{\mathbf{C}}_{t+1} = \xi_t \hat{\mathbf{C}}_t + \zeta_t \mathbf{x}_{t+1} \mathbf{x}_{t+1}^\top$$

- SpatioTemporal coVariance Filter (STVF)

$$\mathbf{z}_t := \mathbf{H}(\hat{\mathbf{C}}_t, \mathbf{h}_t, \mathbf{x}_{T:t}) = \sum_{t'=0}^{T-1} \sum_{k=0}^K h_{kt'} \hat{\mathbf{C}}_t^k \mathbf{x}_{t-t'}$$

⇒ Spatial (graph) and temporal convolutions

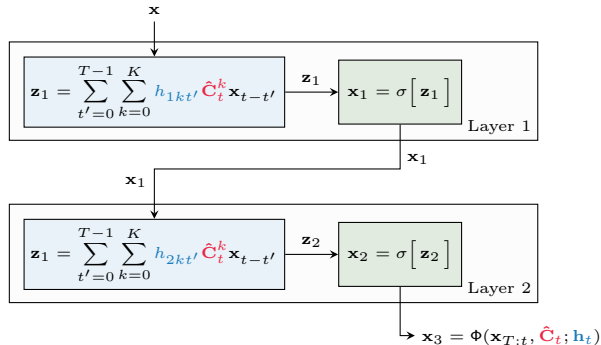


SpatioTemporal Covariance Neural Networks (STVNNs)

► SpatioTemporal coVariance Neural Network (STVNN):

⇒ Sequence of filterbanks followed by non-linearity

$$\mathbf{z}_t^l = \sigma \left(\mathbf{H}^l(\hat{\mathbf{C}}_t, \mathbf{h}_t, \mathbf{z}_{T:t}^{l-1}) \right)$$

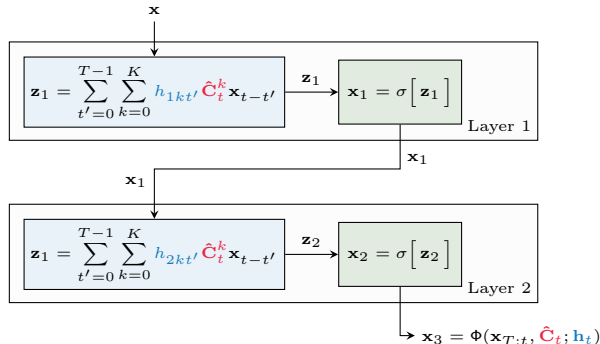


SpatioTemporal Covariance Neural Networks (STVNNs)

► SpatioTemporal coVariance Neural Network (STVNN):

⇒ Sequence of filterbanks followed by non-linearity

$$\mathbf{z}_t^l = \sigma \left(\mathbf{H}^l(\hat{\mathbf{C}}_t, \mathbf{h}_t, \mathbf{z}_{T:t}^{l-1}) \right)$$



► Online parameter updates

$$\mathbf{h}_{t+1} = \mathbf{h}_t - \eta \nabla_t \mathcal{L}(\Phi(\mathbf{x}_{T:t}, \hat{\mathbf{C}}_t; \mathbf{h}_t))$$



Stability of STVNNs

- Stability for **true** $\mathbf{H}(\mathbf{C}, \mathbf{h}^*, \mathbf{x}_{T:t})$ vs. **estimated** $\mathbf{H}(\hat{\mathbf{C}}_t, \mathbf{h}_t, \mathbf{x}_{T:t})$ spatiotemporal online covariance filter:

$$\left\| \mathbf{H}(\hat{\mathbf{C}}_t, \mathbf{h}_t, \mathbf{x}_{T:t}) - \mathbf{H}(\mathbf{C}, \mathbf{h}^*, \mathbf{x}_{T:t}) \right\| \leq \frac{1}{\sqrt{t}} PTN \left(k_{max} e^{\epsilon/2} + QG \|\mathbf{C}\| \sqrt{\log N + u} \right) + \frac{\|\mathbf{h}^*\|^2}{2\eta t} + \mathcal{O} \left(\frac{1}{t} \right)$$

Cavallo, Sabbaqi & Isufi. Spatiotemporal Covariance Neural Networks. ECML PKDD, 2024



Stability of STVNNs

- Stability for **true** $\mathbf{H}(\mathbf{C}, \mathbf{h}^*, \mathbf{x}_{T:t})$ vs. **estimated** $\mathbf{H}(\hat{\mathbf{C}}_t, \mathbf{h}_t, \mathbf{x}_{T:t})$ spatiotemporal online covariance filter:

$$\left\| \mathbf{H}(\hat{\mathbf{C}}_t, \mathbf{h}_t, \mathbf{x}_{T:t}) - \mathbf{H}(\mathbf{C}, \mathbf{h}^*, \mathbf{x}_{T:t}) \right\| \leq \frac{1}{\sqrt{t}} P T N \left(k_{max} e^{\epsilon/2} + Q G \|\mathbf{C}\| \sqrt{\log N + u} \right) + \frac{\|\mathbf{h}^*\|^2}{2\eta t} + \mathcal{O}\left(\frac{1}{t}\right)$$

- P – Lipschitz constant controls stability vs. discriminability trade-off

Cavallo, Sabbaqi & Isufi. Spatiotemporal Covariance Neural Networks. ECML PKDD, 2024



Stability of STVNNs

- Stability for **true** $\mathbf{H}(\mathbf{C}, \mathbf{h}^*, \mathbf{x}_{T:t})$ vs. **estimated** $\mathbf{H}(\hat{\mathbf{C}}_t, \mathbf{h}_t, \mathbf{x}_{T:t})$ spatiotemporal online covariance filter:

$$\left\| \mathbf{H}(\hat{\mathbf{C}}_t, \mathbf{h}_t, \mathbf{x}_{T:t}) - \mathbf{H}(\mathbf{C}, \mathbf{h}^*, \mathbf{x}_{T:t}) \right\| \leq \frac{1}{\sqrt{t}} P \overset{TN}{T} N \left(k_{max} e^{\epsilon/2} + QG \|\mathbf{C}\| \sqrt{\log N + u} \right) + \frac{\|\mathbf{h}^*\|^2}{2\eta t} + \mathcal{O}\left(\frac{1}{t}\right)$$

- P – Lipschitz constant controls stability vs. discriminability trade-off
- T – temporal window size $\rightarrow$ trade-off temporal information vs stability

Cavallo, Sabbaqi & Isufi. Spatiotemporal Covariance Neural Networks. ECML PKDD, 2024



Stability of STVNNs

- Stability for **true** $\mathbf{H}(\mathbf{C}, \mathbf{h}^*, \mathbf{x}_{T:t})$ vs. **estimated** $\mathbf{H}(\hat{\mathbf{C}}_t, \mathbf{h}_t, \mathbf{x}_{T:t})$ spatiotemporal online covariance filter:

$$\left\| \mathbf{H}(\hat{\mathbf{C}}_t, \mathbf{h}_t, \mathbf{x}_{T:t}) - \mathbf{H}(\mathbf{C}, \mathbf{h}^*, \mathbf{x}_{T:t}) \right\| \leq \frac{1}{\sqrt{t}} P T N \left(k_{max} e^{\epsilon/2} + Q G \|\mathbf{C}\| \sqrt{\log N + u} \right) + \frac{\|\mathbf{h}^*\|^2}{2\eta t} + \mathcal{O}\left(\frac{1}{t}\right)$$

- P – Lipschitz constant controls stability vs. discriminability trade-off
- T – temporal window size $\rightarrow$ trade-off temporal information vs stability
- Usual **data dimensionality** N – **number of samples** t trade-off

Cavallo, Sabbaqi & Isufi. Spatiotemporal Covariance Neural Networks. ECML PKDD, 2024

Stability of STVNNs

- Stability for **true** $\mathbf{H}(\mathbf{C}, \mathbf{h}^*, \mathbf{x}_{T:t})$ vs. **estimated** $\mathbf{H}(\hat{\mathbf{C}}_t, \mathbf{h}_t, \mathbf{x}_{T:t})$ spatiotemporal online covariance filter:

$$\left\| \mathbf{H}(\hat{\mathbf{C}}_t, \mathbf{h}_t, \mathbf{x}_{T:t}) - \mathbf{H}(\mathbf{C}, \mathbf{h}^*, \mathbf{x}_{T:t}) \right\| \leq \frac{1}{\sqrt{t}} P T N \left(k_{max} e^{\epsilon/2} + Q G \|\mathbf{C}\| \sqrt{\log N + u} \right) + \frac{\|\mathbf{h}^*\|^2}{2\eta t} + \mathcal{O}\left(\frac{1}{t}\right)$$

- P – Lipschitz constant controls stability vs. discriminability trade-off
- T – temporal window size $\rightarrow$ trade-off temporal information vs stability
- Usual data dimensionality N – number of samples t trade-off
- **Online parameter estimation error disappears faster** ($\mathcal{O}(1/t)$)

Cavallo, Sabbaqi & Isufi. Spatiotemporal Covariance Neural Networks. ECML PKDD, 2024



Stability of STVNNs

- Stability for **true** $\mathbf{H}(\mathbf{C}, \mathbf{h}^*, \mathbf{x}_{T:t})$ vs. **estimated** $\mathbf{H}(\hat{\mathbf{C}}_t, \mathbf{h}_t, \mathbf{x}_{T:t})$ spatiotemporal online covariance filter:

$$\left\| \mathbf{H}(\hat{\mathbf{C}}_t, \mathbf{h}_t, \mathbf{x}_{T:t}) - \mathbf{H}(\mathbf{C}, \mathbf{h}^*, \mathbf{x}_{T:t}) \right\| \leq \frac{1}{\sqrt{t}} PTN \left(k_{max} e^{\epsilon/2} + QG \|\mathbf{C}\| \sqrt{\log N + u} \right) + \frac{\|\mathbf{h}^*\|^2}{2\eta t} + \mathcal{O}\left(\frac{1}{t}\right)$$

- P – Lipschitz constant controls stability vs. discriminability trade-off
- T – temporal window size $\rightarrow$ trade-off temporal information vs stability
- Usual data dimensionality N – number of samples t trade-off
- Online parameter estimation error disappears faster ($\mathcal{O}(1/t)$)
- STVNNs are **stable** to **both** the covariance and parameter estimation errors

Forecasting Performance

► Baselines

⇒ TPCA: temporal PCA

⇒ VNN: VNN with temporal data as features

⇒ VNNL: VNN on each snapshot followed by LSTM for sequence modeling

Datasets	Steps	LSTM	TPCA	VNN	VNNL	STVNN
NOAA	1	1.98	2.42	1.71	<u>1.67</u>	1.35
	3	3.10	3.93	<u>3.14</u>	3.36	3.21
	5	3.46	5.10	4.03	5.76	<u>3.71</u>
Molene	1	0.29	0.35	0.20	0.21	0.20
	3	0.47	0.46	0.43	<u>0.41</u>	0.38
	5	0.60	0.56	0.64	<u>0.59</u>	0.56
Exchange rate	1	1.25	1.73	0.70	<u>0.68</u>	0.65
	3	1.33	1.66	<u>0.98</u>	1.00	0.94
	5	1.39	1.75	1.19	<u>1.16</u>	1.11

► STVNN more consistent across tasks and datasets

► STVNN better than TPCA for stability and than VNN and VNNL for temporal convolution



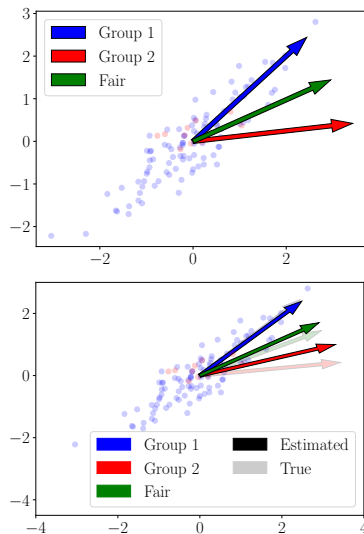
Outline

- ▶ Covariance Neural Networks
- ▶ Sparse VNNs
- ▶ Spatiotemporal VNNs
- ▶ **Fair VNNs**
- ▶ Conclusions



Limited Data as Source of Unfairness

- Datasets might contain harmful biases:
 - ⇒ E.g., underrepresented groups
 - ⇒ VNNs might have unfair performance
 - ⇒ Fair PCA might fail due to instability
- Solution: Fair-VNNs!
 - ⇒ How can VNN's stability help?
 - ⇒ How to make VNNs fair?



Fair Covariance Neural Networks (F-VNNs)

- **Definition:** *equality of performance across groups*



Fair Covariance Neural Networks (F-VNNs)

- **Definition:** *equality of performance across groups*
- **Balanced covariance estimate** for two groups g, h

$$\mathbf{C}_{bal} = \alpha \hat{\mathbf{C}} + (1 - \alpha)(\hat{\mathbf{C}}_h - \hat{\mathbf{C}}_g) = \alpha_g \hat{\mathbf{C}}_g + \alpha_h \hat{\mathbf{C}}_h$$



Fair Covariance Neural Networks (F-VNNs)

- **Definition:** *equality of performance across groups*
- **Balanced covariance estimate** for two groups g, h

$$\mathbf{C}_{bal} = \alpha \hat{\mathbf{C}} + (1 - \alpha)(\hat{\mathbf{C}}_h - \hat{\mathbf{C}}_g) = \alpha_g \hat{\mathbf{C}}_g + \alpha_h \hat{\mathbf{C}}_h$$

- **Bias mitigation penalties**

⇒ F-VNNs are trained with a loss penalty to encourage fairness

$$\min_{\mathcal{H}} \gamma \mathcal{L}(\mathbf{X}, \mathbf{y}, \Phi) + (1 - \gamma) \mathcal{R}(\mathbf{X}, \mathbf{y}, \mathbf{z}, \Phi)$$

⇒ $\mathcal{L}$ is **task-specific loss** (e.g., cross-entropy)

⇒ $\mathcal{R}$ is **bias penalty** (e.g., performance difference across groups)

⇒ γ is **balancing term**



Stability of F-VNNs

► Fair covariance estimates

⇒ $\mathbf{C}_{bal}$ is subject to covariance estimation errors

⇒ Fair PCA is **unstable** → might result in **biased treatment**

► VNNs' stability as fairness

⇒ F-VNNs are stable with fair covariance estimates:

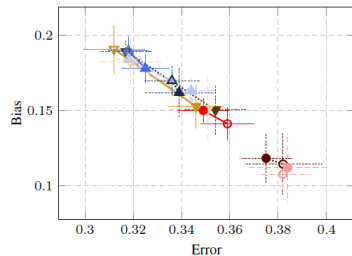
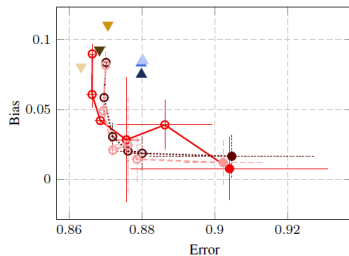
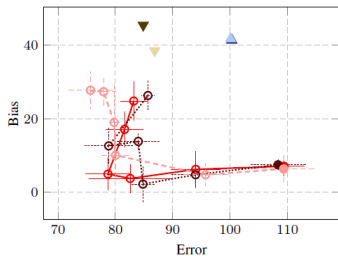
$$\|\mathbf{H}(\mathbf{C}_{bal}) - \mathbf{H}(\mathbf{C})\| \leq P\sqrt{N + 2N^2} \left(\mathcal{O}\left(\frac{1}{\sqrt{T_g}}\right) + \mathcal{O}\left(\frac{1}{\sqrt{T_h}}\right) \right)$$

⇒ **Stability** corresponds to better **fairness** as it guarantees **consistent behavior across groups**



Numerical Results

Dataset	Description	Task	Sensitive attribute
Parkinson (left)	Medical records of patients	Regression for Parkinson's level	Gender
LSAC (center)	Law school students' features	Regression for GPA	Race
German credit (right)	Features of individuals applying for credit	Classification (good or bad)	Gender



- F-VNNs are flexible in the bias vs error tradeoff by changing γ
- Generally, F-VNNs perform better than PCA and improve fairness



Outline

- ▶ Covariance Neural Networks
- ▶ Sparse VNNs
- ▶ Spatiotemporal VNNs
- ▶ Fair VNNs
- ▶ **Conclusions**



Conclusions

- ▶ Learning with covariance matrices

- ⇒ Capture hidden data relations

- ⇒ Unstable and ineffective in low-data high-dimensional regimes



Conclusions

- ▶ Learning with covariance matrices

- ⇒ Capture hidden data relations

- ⇒ Unstable and ineffective in low-data high-dimensional regimes

- ▶ coVariance Neural Networks (VNNs)

- ⇒ Process covariance matrices and **are stable**

- ⇒ Computationally **inefficient** and may **fail** in sparse setting → Sparse VNNs

- ⇒ Cannot handle **non-stationary temporal** data → Spatiotemporal VNNs

- ⇒ **Suffer** data imbalance → Fair VNNs



Conclusions

- ▶ Learning with covariance matrices

- ⇒ Capture hidden data relations

- ⇒ Unstable and ineffective in low-data high-dimensional regimes

- ▶ coVariance Neural Networks (VNNs)

- ⇒ Process covariance matrices and **are stable**

- ⇒ Computationally **inefficient** and may **fail** in sparse setting → **Sparse VNNs**

- ⇒ Cannot handle **non-stationary temporal** data → **Spatiotemporal VNNs**

- ⇒ **Suffer** data imbalance → **Fair VNNs**

- ▶ **Thank you for the attention! Main references and code:**

- ⇒ Sihag, Mateos, McMillan & Ribeiro. coVariance Neural Networks. NeurIPS, 2022

- ⇒ Cavallo, Sabbaqi & I. Spatiotemporal Covariance Neural Networks. ECML PKDD, 2024

- ⇒ Cavallo, Navarro, Segarra & I. Fair Covariance Neural Networks. under review, 2024

